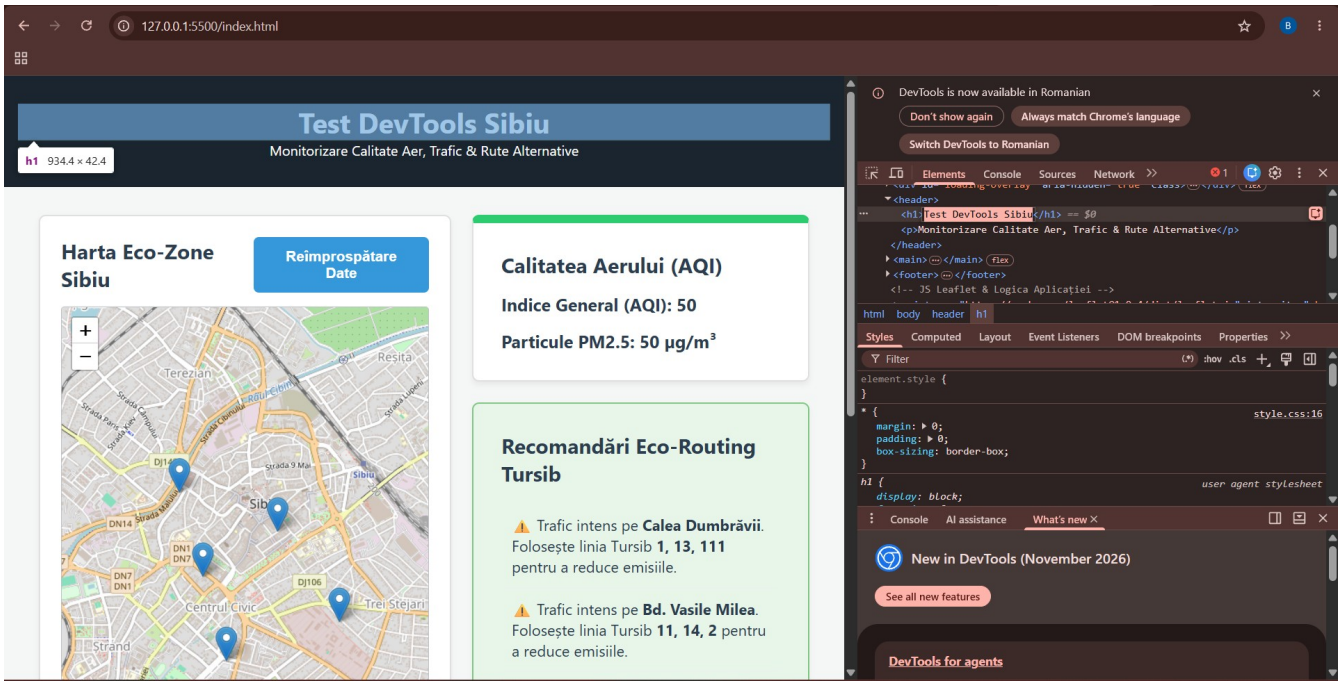
Raport – Analiza aplicației cu Developer Tools

Tehnologii Web Avansate | Laborator de 80 de minute

Date student	De completat
Nume și prenume	**Roman Bogdan-Constantin**
Aplicație / API folosit	**Sibiu AirQuality & Eco-Traffic Tracker, conectat la WAQI API (World Air Quality Index)**
Repository GitHub	**https://github.com/RomanBogdanC/sibiu-eco-tracker**
Browser / mod de rulare	**Google Chrome / http://127.0.0.1:5500/index.html**

Instrucțiuni: Înlocuiți marcajele [Captură] cu capturile făcute în timpul laboratorului. Lăsați sub fiecare 1–2 propoziții care explică ce se vede și ce ați constatat. Nu includeți informații sensibile.

1. Modificarea DOM în Elements

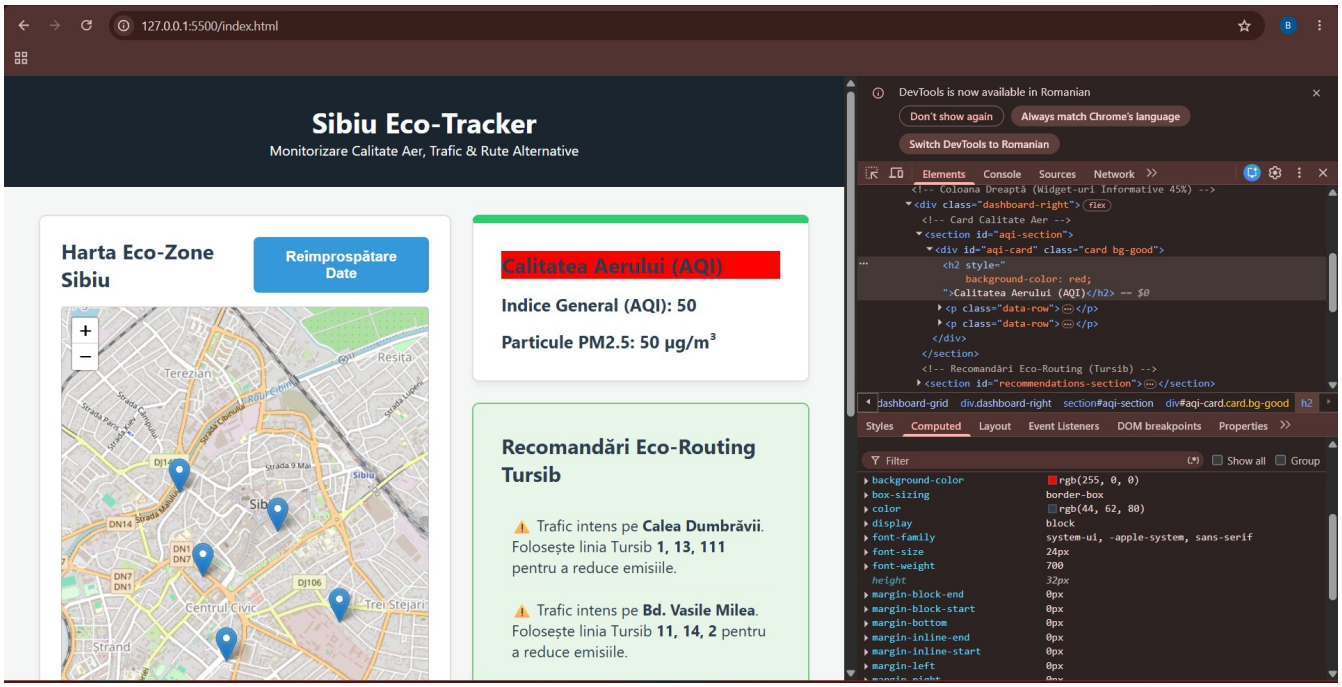


Arătați o modificare temporară de text sau structură, vizibilă în pagină.

Elementul modificat: **Titlul principal <h1> din sectiunea <header>.**

Ce s-a întâmplat după reload? **Modificarea a dispărut și a revenit textul original, deoarece schimbările făcute în DevTools se aplică temporar doar în memoria browserului, nu suprascriu fișierul sursă `index.html`.**

2. Styles / Computed / Box model

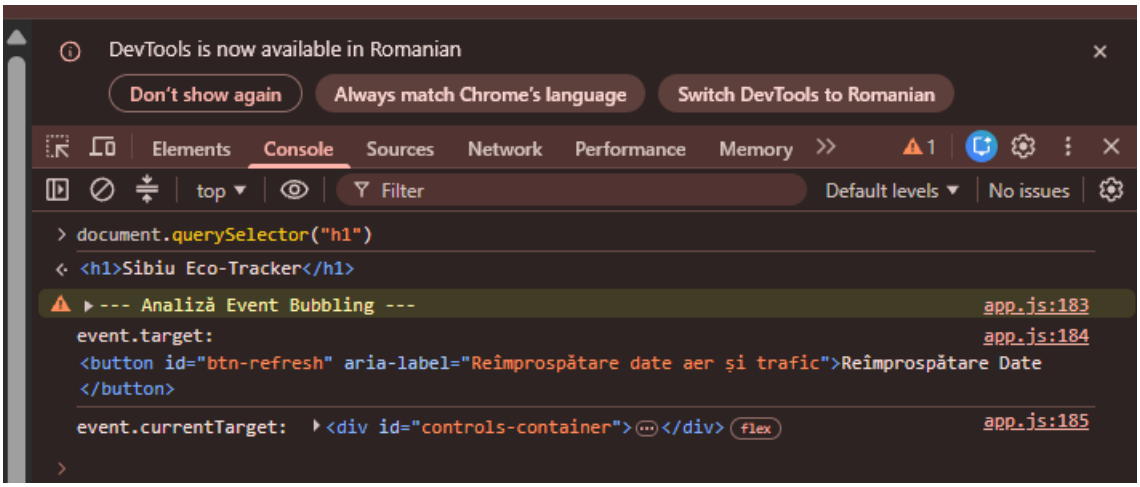


Arătați una sau mai multe reguli editate și diagrama box model.

Proprietăți CSS modificate: **Am modificat inline proprietatea background-color steand-o pe red.**

Ce valori ați observat pentru padding/border/margin/box-sizing? **padding: 32 pe toate laturile; border: 9600 sus, 1600 in rest; margin: 0 pe toate laturile; box-sizing: border-box**

3. Console – expresii și evenimente

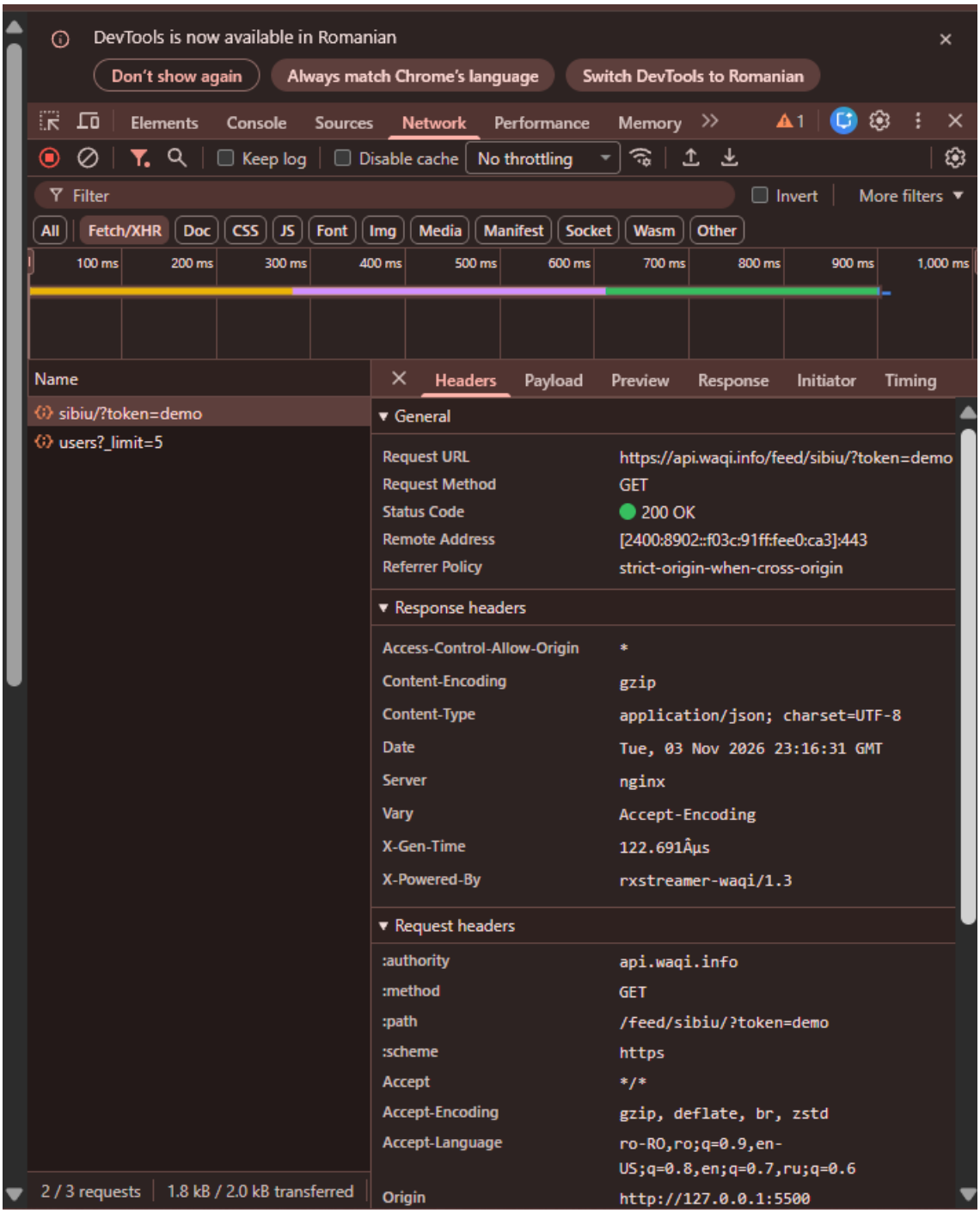


Arătați rezultatul unei expresii sau mesaje event.target/event.currentTarget

Ce ați testat în Console? **Am testat selectarea titlului <h1> și am declanșat evenimentul de click pe butonul de refresh pentru a analiza propagarea evenimentelor (Event Bubbling).**

Care este diferența dintre target și currentTarget? **Event.target reprezintă elementul exact pe care s-a dat click (butonul efectiv #btn-refresh), iar event.currentTarget este elementul de care este atașat event listener-ul (containerul #controls-container).**

4. Network – cererea către API



Selectați o cerere Fetch/XHR și arătați Headers.

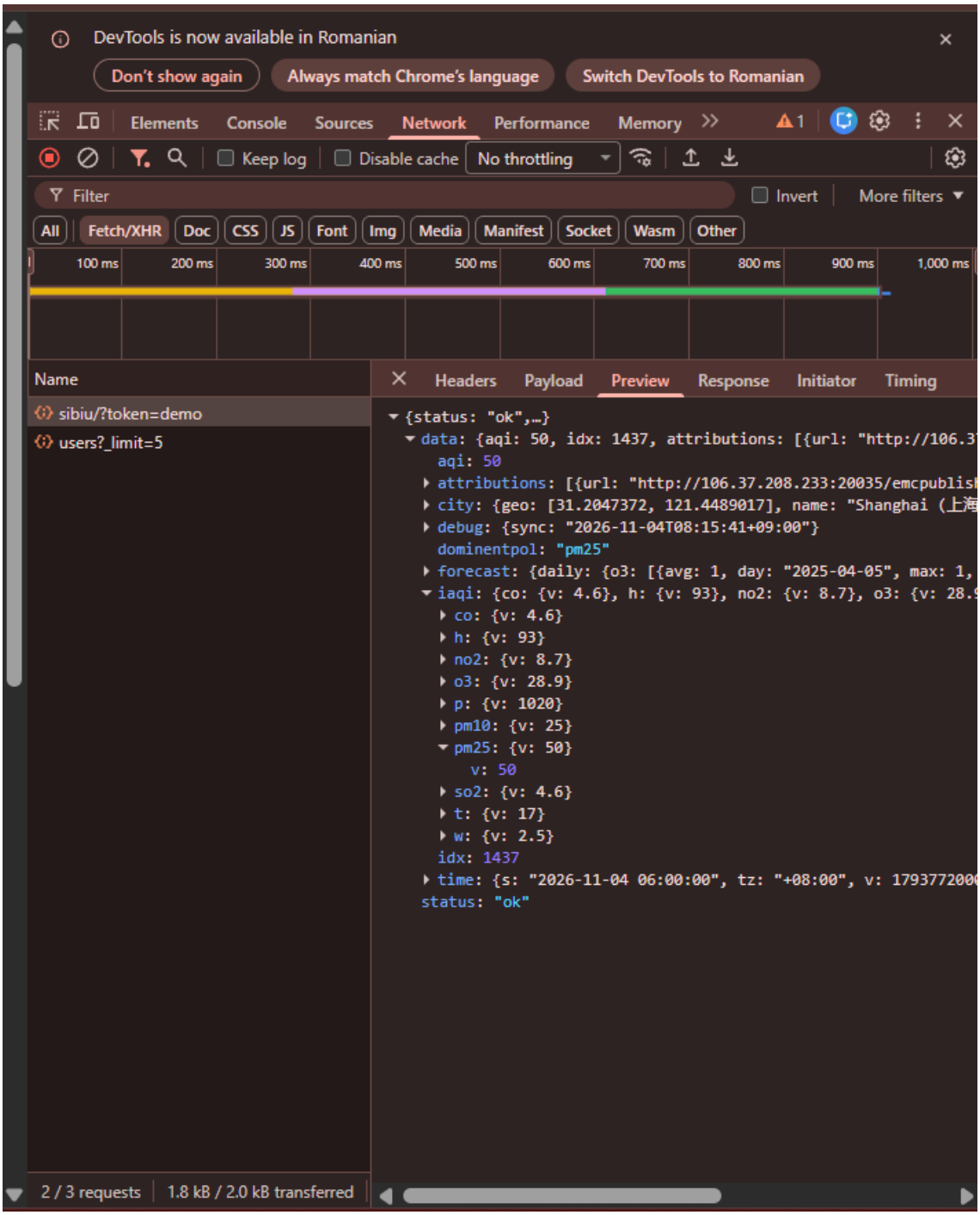
Metodă HTTP: **GET**

Status HTTP: **200 OK**

Durata normală (ms): **aprox 900ms**

Ce a declanșat cererea? **Încărcarea paginii (evenimentul DOMContentLoaded) sau apăsarea butonului „Reîmprospătare Date", care a apelat funcția fetch() in JavaScript.**

5. Network – răspunsul API

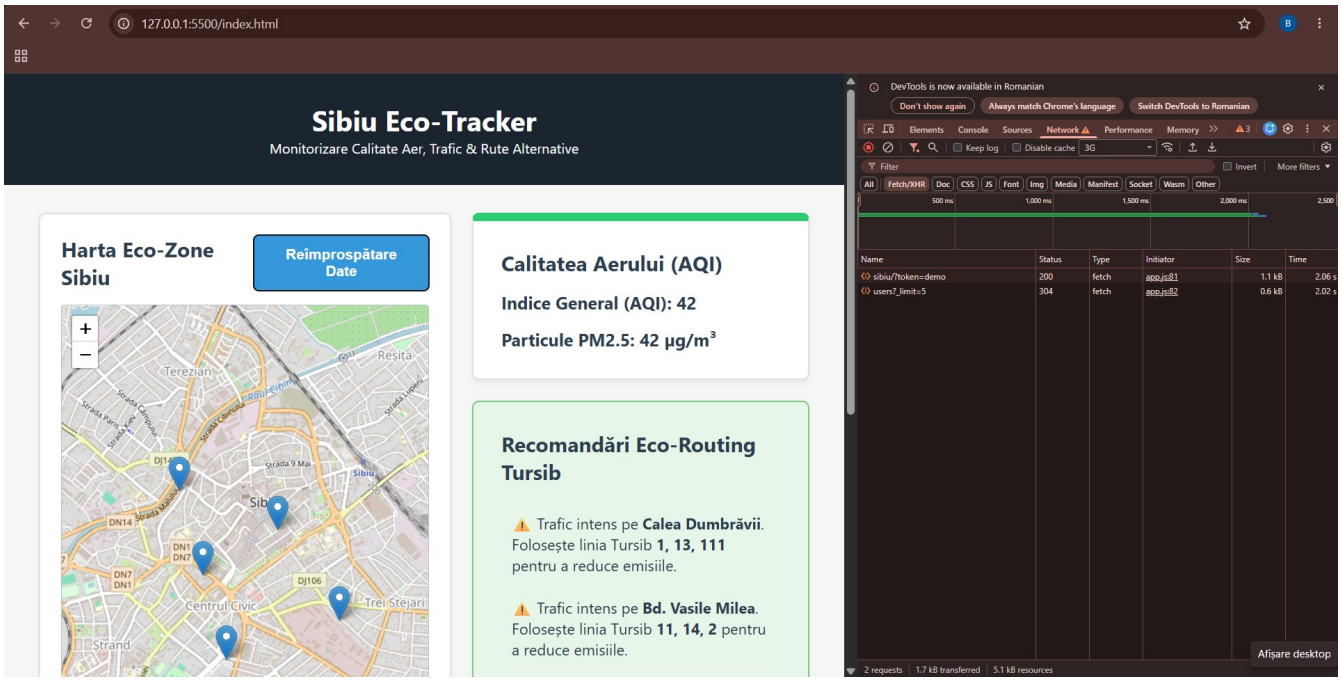


Arătați Response/Preview pentru aceeași cerere, cu două câmpuri JSON.

Câmp JSON 1 și utilizarea sa: **data.aqi (cu val 50): acesta reprezintă indicele de calitate a aerului, pe care îl folosesc pentru a afișa valoarea numerică în interfață și pentru a schimba dinamic clasa CSS a cardului.**

Câmp JSON 2 și utilizarea sa: data.iaqi.pm25.v (cu val 50): **acest câmp indică nivelul particulelor fine PM2.5 și este extras pentru a fi afișat în textul detaliat al cardului despre calitatea aerului.**

6. Network – conexiune lentă



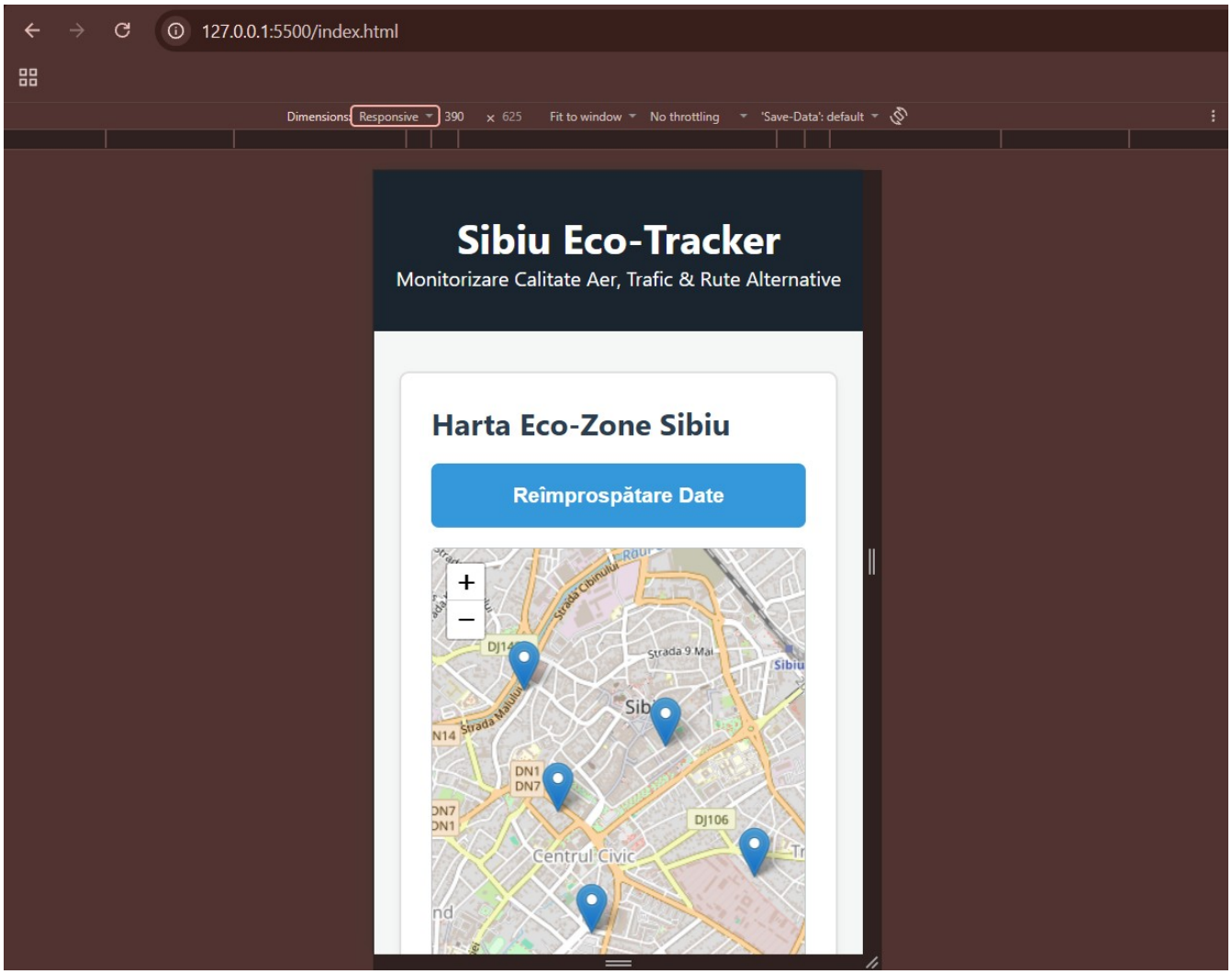
Arătați throttling-ul activat și aceeași cerere repetată.

Durata cu limitare (ms): **aprox 2060ms**

Diferența față de testul normal : **a crescut semnificativ (initial fiind sub 1s)**

Afișează aplicația loading? **Da, pe durata așteptării acestor 2 secunde, interfața a fost acoperită de un overlay opac cu mesajul "Încărcare date Sibiu…".**

7. Responsive – Device Mode

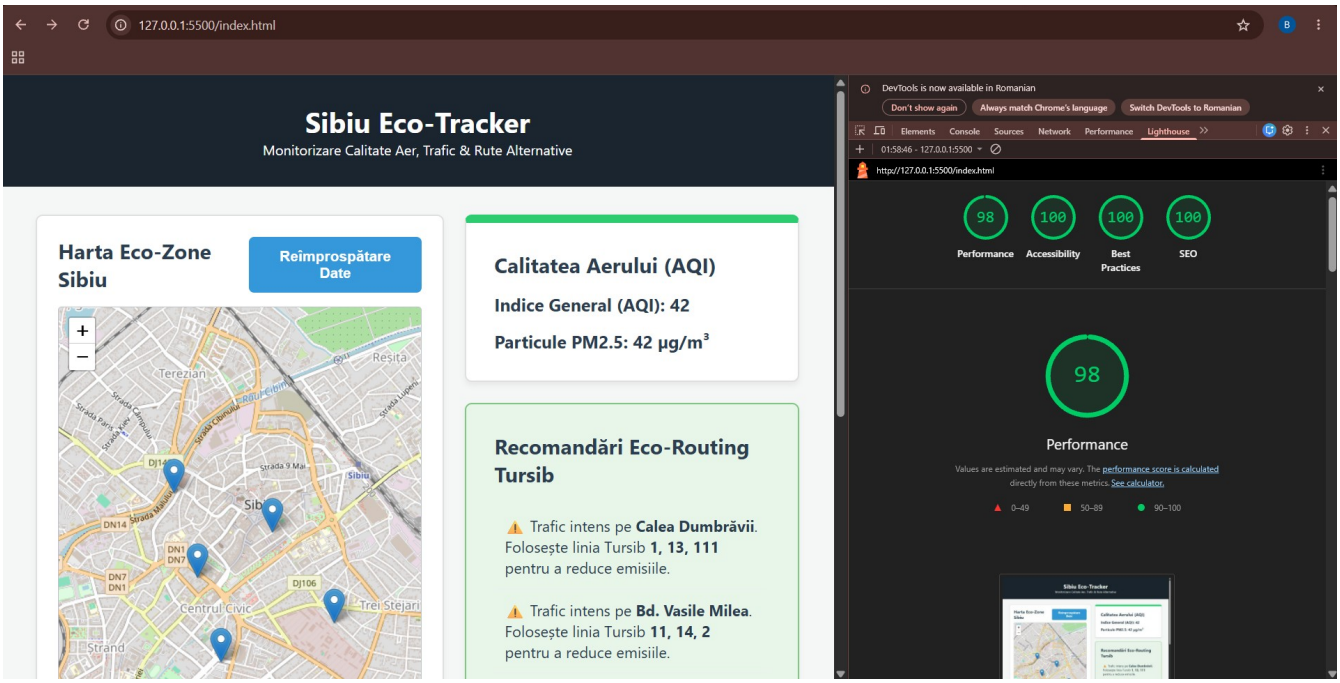


Arătați pagina la circa 390 px lățime.

Lățimea de test: **390px**

Problemă observată sau motivul pentru care layoutul este utilizabil: **Layout-ul este perfect utilizabil deoarece, datorită regulilor CSS de tip *media query*, design-ul pe două coloane s-a transformat automat într-o singură coloană. Titlul, harta interactivă și butonul de „Reîmprospătare Date” se încadrează corect pe lățimea ecranului, fără a genera scroll orizontal nedorit, oferind o experiență optimă pe mobil.**

8. Lighthouse – audit



Arătați raportul cu categoriile și sumarizați 1–2 recomandări.

Performance: **98**

Accessibility: **100**

Best Practices: **100**

SEO: **100**

Recomandarea aleasă: **"Render-blocking requests". Auditul estimează o economie de timp de aproximativ 290 ms.**

Cum ați remedia-o? Această problemă apare deoarece fișierele externe de CSS și JavaScript (în special cele de la Leaflet.js incluse în <head>) blochează randarea inițială a paginii. Aș remedia-o adăugând atributul defer sau async la scripturile externe și, pentru CSS, aș putea extrage "critical CSS" direct în <style> pentru încărcarea inițială, lăsând restul să se încarce asincron.

Concluzie (max. 5 rânduri)

Ce ați aflat despre aplicația voastră? Care este primul lucru pe care l-ați îmbunătăți în cod?

Am constatat că aplicația dezvoltată exclusiv cu tehnologii web native (Vanilla JS, HTML semantic, CSS pur) este extrem de performantă, obținând scoruri aproape maxime în Lighthouse. Modificările asincrone (fetch API) și manipularea DOM-ului se realizează instantaneu, iar layout-ul este perfect adaptabil. Primul lucru pe care l-aș îmbunătăți în cod ar fi implementarea atributului defer pentru scripturile externe, pentru a elimina complet blocajele de randare (render-blocking requests) și a atinge scorul perfect de 100 la performanță.

Predare

Salvați acest document în docs/ din repository-ul proiectului, efectuați commit și push, apoi transmiteți linkul repository-ului.